

Cahier des charges – Projet TaskSync

1. Objectif du projet

Développer une **application de gestion de tâches multiplateforme** permettant aux utilisateurs de créer, organiser, suivre et synchroniser des tâches entre une **interface Web** (HTML/CSS + TypeScript) et une **application desktop** (Qt en C++), à l'aide d'une **API REST** centrale (Python avec FastAPI).

2. Utilisateurs visés

- Étudiants en projet de groupe
 - Développeurs ou freelances gérant plusieurs projets
 - Petites équipes collaborant à distance
 - Toute personne souhaitant gérer ses tâches personnelles ou professionnelles
-

3. Fonctionnalités principales

Authentification

- Inscription / Connexion
- Gestion de session (jetons d'authentification)

Gestion de projets

- Créer un projet
- Supprimer ou modifier un projet
- Lister les projets de l'utilisateur

Gestion des tâches

- Ajouter une tâche à un projet
- Modifier / supprimer une tâche
- Attribuer une tâche à un utilisateur
- Marquer une tâche comme terminée
- Filtrer / trier les tâches (par état, priorité, échéance)

Synchronisation

- L'application Qt peut fonctionner hors ligne

- Synchronisation automatique avec l'API dès qu'une connexion est disponible
- Gestion des conflits simples

4. 🖥️ Plateformes et technologies prévues

Composant	Technologie prévue
Interface Web	HTML, CSS, TypeScript
Interface Desktop	Qt (C++)
API REST	Python (FastAPI)
Base de données	SQLite (ou PostgreSQL plus tard)
Échange de données JSON via HTTP (API REST)	
Authentification	JWT Tokens (auth sécurisée)
Versionnement	Git + GitHub/GitLab (optionnel)

5. 🛡️ Contraintes techniques

- Fonctionnement hors ligne de l'application Qt (stockage local temporaire)
- Connexion sécurisée entre les composants
- Conformité API REST (architecture propre et extensible)
- Application modulaire pour permettre des ajouts futurs

6. 📅 Plan de développement (résumé)

Étape	Objectif principal
1. Analyse & conception	Définir fonctionnalités, schémas UML, API
2. Implémentation de l'API REST	Routes, logique métier, base de données
3. Développement interface Web	Connexion à l'API, affichage et manipulation
4. Développement application Qt	Interface, stockage local, synchro
5. Tests et robustesse	Cas limites, erreurs, sécurité

Étape**Objectif principal**

6. Déploiement et documentation Lancer le système localement / en ligne